TUTORIAL – 15

Automated Login Testing

Aim:

To create automated login functionality, authentication validation, assertions and exception handling scenarios using Selenium WebDriver with Python.

Description:

Authentication testing verifies whether valid users can access a system and invalid users are properly rejected.

In this practical, Selenium WebDriver with Python is used to automate:

- Valid login
- Invalid password
- Invalid username
- Empty credentials
- Authentication assertions
- Error message validation
- Logout
- Exception handling

Problem Statement:

Create an automated Selenium login testing script that performs:

1. Valid login
2. Invalid password testing
3. Invalid username testing
4. Empty credential testing
5. Authentication validation
6. Assertions
7. Exception handling
8. Logout

Implementation Details

Parameter	Details
Project Name	Automated Login Testing
Application/Software	SauceDemo
Module Name	Authentication/Login
SDLC Model	Agile

Programming Language	Python
IDE/Tool Used	PyCharm
Automation Tool	Selenium WebDriver
Browser	Google Chrome
Input/Test Data	Valid and Invalid Login Credentials

Automation Environment Configuration

Parameter	Details
Python Version	Python 3.x
Selenium Version	Installed using pip
Browser	Google Chrome
WebDriver	Selenium-managed Chrome WebDriver
IDE Used	PyCharm
Target URL	https://www.saucedemo.com/

Login Automation Scenario

Scenario ID	Scenario	Expected Result
TC-01	Valid Login	Login successful
TC-02	Invalid Password	Error message displayed
TC-03	Invalid Username	Error message displayed
TC-04	Empty Credentials	Validation message displayed
TC-05	Logout	User returns to login page

Procedure – How to Perform

Step 1 – Create Project

Open PyCharm.

Create:

SoftwareTesting

Step 2 – Create File
Create:
tutorial15.py

Step 3 – Install Selenium
Open PyCharm Terminal:
pip install selenium

Step 4 – Import Selenium
```
from selenium import webdriver
from selenium.webdriver.common.by import By
```

Step 5 – Import Wait and Exceptions
```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

from selenium.common.exceptions import (
    NoSuchElementException,
    TimeoutException,
    ElementNotInteractableException,
    StaleElementReferenceException
)
```

Step 6 – Create Browser
```
driver = webdriver.Chrome()
driver.maximize_window()
```

Step 7 – Perform Valid Login
Username: standard_user
Password: secret_sauce

Step 8 – Validate Successful Login
```
assert "inventory" in driver.current_url
```

Step 9 – Test Invalid Password
Use:
Username: standard_user
Password: wrong_password

Verify error message.

Step 10 – Test Invalid Username

Use:

Username: wrong_user

Password: secret_sauce

Verify error message.

Step 11 – Test Empty Credentials

Leave username and password blank and click Login.

Verify:

Username is required

Step 12 – Logout

Login successfully, open menu and click Logout.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

from selenium.common.exceptions import (
    NoSuchElementException,
    TimeoutException,
    ElementNotInteractableException,
    StaleElementReferenceException
)


URL = "https://www.saucedemo.com/"

VALID_USERNAME = "standard_user"
VALID_PASSWORD = "secret_sauce"


def start_browser():

    driver = webdriver.Chrome()
    driver.maximize_window()

    return driver
```

```
def login(driver, username, password):

    driver.get(URL)

    wait = WebDriverWait(driver, 10)

    username_box = wait.until(
        EC.visibility_of_element_located(
            (By.ID, "user-name")
        )
    )

    password_box = wait.until(
        EC.visibility_of_element_located(
            (By.ID, "password")
        )
    )

    login_button = wait.until(
        EC.element_to_be_clickable(
            (By.ID, "login-button")
        )
    )

    username_box.clear()
    username_box.send_keys(username)

    password_box.clear()
    password_box.send_keys(password)

    login_button.click()


# TC-01 Valid Login
def test_valid_login():

    driver = start_browser()
```

```
try:

    login(
        driver,
        VALID_USERNAME,
        VALID_PASSWORD
    )

    WebDriverWait(driver, 10).until(
        EC.url_contains("inventory")
    )

    assert "inventory" in driver.current_url

    products_title = WebDriverWait(
        driver, 10
    ).until(
        EC.visibility_of_element_located(
            (By.CLASS_NAME, "title")
        )
    )

    assert products_title.text == "Products"

    print("TC-01 Valid Login: PASS")
    print("Authentication Assertion: PASS")

except AssertionError:
    print("TC-01 Valid Login: FAIL")

except NoSuchElementException:
    print("NoSuchElementException handled")

except TimeoutException:
    print("TimeoutException handled")

except ElementNotInteractableException:
    print(
        "ElementNotInteractableException handled"
    )
```

```
    except StaleElementReferenceException:
        print(
            "StaleElementReferenceException handled"
        )

    finally:
        driver.quit()


# TC-02 Invalid Password
def test_invalid_password():

    driver = start_browser()

    try:

        login(
            driver,
            VALID_USERNAME,
            "wrong_password"
        )

        error_message = WebDriverWait(
            driver, 10
        ).until(
            EC.visibility_of_element_located(
                (
                    By.CSS_SELECTOR,
                    "h3[data-test='error']"
                )
            )
        )

        assert (
            "Username and password do not match"
            in error_message.text
        )

        print("TC-02 Invalid Password: PASS")
```

```
    except AssertionError:
        print("TC-02 Invalid Password: FAIL")

    except TimeoutException:
        print("Error message was not displayed")

    finally:
        driver.quit()


# TC-03 Invalid Username
def test_invalid_username():

    driver = start_browser()

    try:

        login(
            driver,
            "wrong_user",
            VALID_PASSWORD
        )

        error_message = WebDriverWait(
            driver, 10
        ).until(
            EC.visibility_of_element_located(
                (
                    By.CSS_SELECTOR,
                    "h3[data-test='error']"
                )
            )
        )

        assert (
            "Username and password do not match"
            in error_message.text
        )
```

```
        print("TC-03 Invalid Username: PASS")

    except AssertionError:
        print("TC-03 Invalid Username: FAIL")

    except TimeoutException:
        print("Error message was not displayed")

    finally:
        driver.quit()


# TC-04 Empty Credentials
def test_empty_credentials():

    driver = start_browser()

    try:

        driver.get(URL)

        login_button = WebDriverWait(
            driver, 10
        ).until(
            EC.element_to_be_clickable(
                (By.ID, "login-button")
            )
        )

        login_button.click()

        error_message = WebDriverWait(
            driver, 10
        ).until(
            EC.visibility_of_element_located(
                (
                    By.CSS_SELECTOR,
                    "h3[data-test='error']"
                )
            )
```

```
        )

        assert (
            "Username is required"
            in error_message.text
        )

        print("TC-04 Empty Credentials: PASS")

    except AssertionError:
        print("TC-04 Empty Credentials: FAIL")

    except TimeoutException:
        print("Validation message was not displayed")

    finally:
        driver.quit()


# TC-05 Logout
def test_logout():

    driver = start_browser()

    try:

        login(
            driver,
            VALID_USERNAME,
            VALID_PASSWORD
        )

        WebDriverWait(driver, 10).until(
            EC.url_contains("inventory")
        )

        menu_button = WebDriverWait(
            driver, 10
        ).until(
            EC.element_to_be_clickable(
```

```
            (By.ID, "react-burger-menu-btn")
        )
    )

    menu_button.click()

    logout_button = WebDriverWait(
        driver, 10
    ).until(
        EC.element_to_be_clickable(
            (By.ID, "logout_sidebar_link")
        )
    )

    logout_button.click()

    WebDriverWait(driver, 10).until(
        EC.visibility_of_element_located(
            (By.ID, "login-button")
        )
    )

    assert "saucedemo.com" in driver.current_url

    print("TC-05 Logout: PASS")

except AssertionError:
    print("TC-05 Logout: FAIL")

except TimeoutException:
    print("Logout operation timed out")

finally:
    driver.quit()


# Execute all test cases

test_valid_login()
test_invalid_password()
```

test_invalid_username()
test_empty_credentials()
test_logout()

Test Case Detailed

Field	Details
Test Case ID	TC-15-01
Requirement ID	REQ-LOGIN-01
Module Name	Authentication
Feature Under Test	Login
Test Scenario	Valid Login
Test Objective	Verify authentication
Test Type	System
Test Technique	Black Box
Priority	High
Severity	Critical
Preconditions	Application available
Postconditions	User logged in
Test Environment	Windows + Python + PyCharm
Browser/OS	Chrome / Windows
Test Data	standard_user / secret_sauce
Expected Result	User successfully logs in
Actual Result	User successfully logged in
Status	Pass
Executed By	_____

Execution Date	_____

Test Case Execution

Step No.	Test Steps	Input	Expected Result	Actual Result	Status
1	Open login page	URL	Login page opens	Opened	Pass
2	Enter username	standard_user	Username accepted	Accepted	Pass
3	Enter password	secret_sauce	Password accepted	Accepted	Pass
4	Click Login	Click	Products page opens	Opened	Pass
5	Validate login	URL/title	Authentication successful	Successful	Pass
6	Invalid password	wrong_password	Error displayed	Displayed	Pass
7	Invalid username	wrong_user	Error displayed	Displayed	Pass
8	Empty credentials	Blank	Validation displayed	Displayed	Pass
9	Logout	Click Logout	Login page displayed	Displayed	Pass

Web Element Identification Table

Element Name	Locator Type	Locator Value	Purpose
Username	ID	user-name	Enter username
Password	ID	password	Enter password
Login Button	ID	login-button	Submit login
Error Message	CSS Selector	h3[data-test='error']	Validate error
Menu Button	ID	react-burger-menu-btn	Open menu
Logout Button	ID	logout_sidebar_link	Logout

Assertion Validation Report

Assertion	Expected Result	Actual Result	Status
Page Title	Products	Products	Pass
URL	inventory URL	inventory URL	Pass
Welcome Message	Product page displayed	Product page displayed	Pass
Login Success	Authentication successful	Successful	Pass
Error Message	Correct error displayed	Correct error displayed	Pass

Exception Handling Report

Exception	Possible Cause	Handled	Remarks
NoSuchElementException	Element cannot be located	Yes	Exception handled
TimeoutException	Element does not appear within wait time	Yes	Timeout handled
ElementNotInteractableException	Element cannot be interacted with	Yes	Exception handled
StaleElementReferenceException	DOM changed after element was located	Yes	Exception handled

Automation Execution Summary

Metric	Value
Total Test Cases	5
Executed	5
Passed	5
Failed	0
Assertions Passed	5+
Exceptions Handled	As encountered

Overall Status	PASS

Observation

Sr. No.	Observation	Remarks
1	Valid credentials authenticated the user	Pass
2	Invalid password was rejected	Pass
3	Invalid username was rejected	Pass
4	Empty credentials generated validation	Pass
5	Logout returned user to login page	Pass

Output

Students should capture screenshots of:

1. PyCharm source code
2. Login page
3. Valid login result
4. Invalid password error
5. Invalid username error
6. Empty credential validation
7. Logout result
8. PyCharm console output

Questionnaire

Q1. What is Authentication Testing?

Answer: Authentication testing verifies whether valid users are allowed to access the application and invalid users are rejected.

Q2. What is Selenium WebDriver?

Answer: Selenium WebDriver is an API used to automate and control web browsers.

Q3. What is Assertion?

Answer: Assertion compares the expected result with the actual result and determines whether the test condition is passed or failed.

Example:

assert "inventory" in driver.current_url

Q4. What is Exception Handling?

Answer: Exception handling is a programming technique used to handle runtime errors without abruptly stopping the program.
Example:

```
try:
    # Selenium operation
except TimeoutException:
    print("Timeout occurred")
```

Q5. Which exception occurs when an element cannot be found?
Answer: NoSuchElementException

IMPORTANT STUDENT EXECUTION CHECKLIST
Tutorial 11
☐ Create tutorial11.py in PyCharm
☐ Install Selenium
☐ Open Chrome using Selenium
☐ Perform login
☐ Navigate to products
☐ Add product
☐ Open cart
☐ Complete checkout form
☐ Validate multi-page workflow
☐ Execute regression scenario
☐ Capture output screenshots
Tutorial 12
☐ Analyze requirement using Generative AI
☐ Generate test cases
☐ Generate test data
☐ Perform defect prediction
☐ Generate/review Selenium code
☐ Execute final code in PyCharm
☐ Prepare regression test cases
☐ Capture AI + PyCharm + browser evidence
Tutorial 13
☐ Open PyCharm
☐ Configure Python interpreter
☐ Install Selenium
☐ Create Python file
☐ Execute Selenium program

☐ Launch Chrome
☐ Verify website
☐ Capture console output

Tutorial 14

☐ Open website
☐ Handle login form
☐ Enter username
☐ Enter password
☐ Click login
☐ Validate URL
☐ Select product
☐ Open cart
☐ Validate result
☐ Capture screenshots

Tutorial 15

☐ Valid Login
☐ Invalid Password
☐ Invalid Username
☐ Empty Credentials
☐ Assertions
☐ Error Message Validation
☐ Exception Handling
☐ Logout
☐ Execute all test cases
☐ Capture PyCharm console output

COMMON SOFTWARE REQUIREMENT FOR ALL PRACTICALS 11–15

Software/Tool	Requirement
IDE	PyCharm
Programming Language	Python
Automation Library	Selenium
Browser	Google Chrome
Testing Tool	Selenium WebDriver
Additional Testing Software	**Not Required**
Bugzilla	**Not Required**

TestLink	**Not Required**
VS Code	**Not Required**
Execution Method	Run .py file from PyCharm

Important: Students should use the faculty GitHub repositories shared by the faculty **only as a reference for understanding the procedure and implementation**. Do **not directly copy-paste** code, reports, screenshots, or outputs. Students must execute the practical themselves and submit their own screenshots and results.